

Can You See Me, GenAI? Characterizing Video-Based Conversational Interaction with Generative AI

Ruizhi Cheng
Meta Platforms, Inc. and
George Mason University
Menlo Park, CA, USA
ruizhicheng@meta.com

Guowu Xie
Meta Platforms, Inc.
Menlo Park, CA, USA
woo@meta.com

Bo Han
George Mason University
Fairfax, VA, USA
bohan@gmu.edu

Abstract

Video-based conversational interaction between humans and generative artificial intelligence (GenAI) represents a rapidly emerging paradigm at the intersection of networking, multimedia communication, and vision language models (VLMs). Applications such as ChatGPT and Gemini enable users to point their smartphones at objects/scenes and receive contextual responses, opening new possibilities for on-the-go visual understanding. However, despite their growing adoption, the underlying performance dynamics and system behavior of these applications remain poorly understood. In this paper, we present the first systematic measurement study of video-based human-GenAI interaction across five representative applications with 11 distinct variants. We develop a custom toolset to characterize these applications across diverse network conditions and interaction scenarios, exposing how design decisions in video delivery and transport protocols shape user-perceived experience. Our measurements reveal several fundamental bottlenecks: conversational latency remains above natural turn-taking thresholds even in unconstrained networks; protocol choices significantly affect system robustness under packet loss; and conventional video streaming practices, such as prioritizing stable frame rates, often degrade VLM inference accuracy. Our findings provide an empirical foundation for designing and optimizing next-generation real-time human-AI interaction.

CCS Concepts

• Information systems → Multimedia streaming; • Computing methodologies → Artificial intelligence.

Keywords

Network Measurement, Human-AI Interaction, Real-time Video Communication, Vision Language Model (VLM), Multimodal Conversational AI

ACM Reference Format:

Ruizhi Cheng, Guowu Xie, and Bo Han. 2026. Can You See Me, GenAI? Characterizing Video-Based Conversational Interaction with Generative AI. In *Proceedings of the 2026 ACM Internet Measurement Conference (IMC '26)*, October 12–16, 2026, Karlsruhe, Germany. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3777912.3839793>



This work is licensed under a Creative Commons Attribution 4.0 International License. *IMC '26, Karlsruhe, Germany*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2327-8/2026/10
<https://doi.org/10.1145/3777912.3839793>

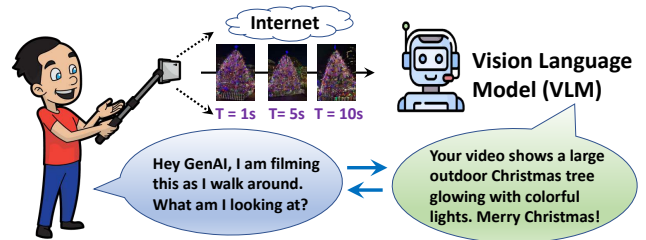


Figure 1: Illustration of video-based conversational interaction between a user and GenAI.

1 Introduction

The emergence of generative artificial intelligence (GenAI) agents empowered by multimodal foundation models marks a major inflection point in human-computer interaction. Recent advances now allow applications such as ChatGPT [48] and Gemini [23] to process and interpret real-time video input, effectively turning everyday mobile devices into powerful platforms for conversational human-GenAI interaction. As shown in Figure 1, these applications follow a common pipeline: the client continuously captures and encodes live camera frames, transmits them over the Internet to the provider's backend, where a vision language model (VLM) [73] performs inference and generates a response that is delivered back to the user, often in the form of synthesized speech.

As conversational video interaction with GenAI becomes increasingly woven into daily mobile experiences, its potential to augment human perception and decision-making continues to expand. For instance, by recognizing objects [72], describing scenes [2], and extracting visual information [50] based on the camera feed of mobile devices, these GenAI services offer immediate, context-aware responses to natural language prompts. This new capability expands the landscape of practical use cases, ranging from assistive tools that support people with visual impairments [47] and real-time visual tutoring and language learning [65], to intelligent productivity assistants that help users better understand and act upon their physical environment [41].

However, delivering such functionality seamlessly requires tightly coordinating live video capture and processing, network transport, and GenAI model inference, whose interactions introduce new design and performance challenges not seen in earlier research on video streaming/analytics [36, 40], real-time communication (RTC) [45], and VLMs/large language models (LLMs) [73]. For instance, unlike traditional interactive video systems such as video conferencing [6, 40] and cloud gaming [28], where user experience

is primarily driven by human-perceived visual quality and responsiveness, video-based human-GenAI interaction also depends critically on whether the backend VLM can correctly understand the streamed visual content and generate accurate responses. Achieving responsive, reliable, and widely accessible experiences, therefore, demands a deep understanding of how these GenAI services operate in realistic conditions.

To shed light on how today's video-based human-GenAI interaction behaves in practice, we conduct a comprehensive measurement study of the end-to-end operation across five leading applications: **ChatGPT** [48] from OpenAI, **Doubao** [4] from ByteDance, **Gemini** [23] from Google, **Grok** [67] from xAI, and **Yuanbao** [56] from Tencent. We develop a custom toolset to conduct controlled experiments across diverse network conditions and interaction scenarios, with a total of 47,750 sessions encompassing 1,030 hours of experiments (§3). Our measurement framework enables us to quantify conversational latency and VLM inference accuracy, capture transport/application-layer behavior, characterize video frame-rate and quality adaptation, and evaluate the robustness of different protocol choices under network stress. The resulting dataset and analysis reveal fundamental limitations in the current design of these applications.

We summarize our key findings as follows.

Finding 1: High Conversational Latency even in Unthrottled Network Environments. Our measurements show that, even under unconstrained network conditions, the user-perceived end-to-end delay consistently exceeds 2s across all applications, with **Doubao** surpassing 6s at the 95th percentile. These delays are far above the ~1s threshold required for natural turn-by-turn conversation [10], despite all applications assigning geographically close entry servers to minimize the length of the bottleneck Internet path. When the network is disrupted (e.g., experiencing packet loss or limited available bandwidth), latencies increase further across all applications. Such elevated delays substantially degrade conversational fluidity and overall user experience.

Finding 2: Limited Inference Capability and Modest User Experience. Whether the underlying VLM can accurately interpret a user's streamed video and audio prompt is another critical determinant of user experience. To quantitatively measure this, we leverage MVBench [34], a well-established benchmark that offers multiple-choice QA tasks for evaluating inference accuracy. Even under unconstrained networks, all applications achieve accuracy below 75%, with the free tiers of **Gemini** and **Grok** falling below 50%. Consistent with these results, the user experience, measured by mean opinion score (MOS), remains modest. On a five-point scale, all applications have average MOS below 4, and the lowest (the free tier of **Grok**) is around only 2. Our measurements further reveal that these key quality of experience (QoE) metrics, accuracy and latency, are strongly influenced by how video is delivered to the backend VLM, including the choice of transport protocol and transmission scheme, as well as the video stream's frame rate and quality.

Finding 3: Protocol and Transmission Choices Affect Networking Performance. Applications adopt different protocol stacks for uplink video delivery. **ChatGPT** and **Yuanbao** employ UDP/RTP at the transport/application layer for frame-based transmission, sending each live-captured video frame individually. **Doubao** similarly relies on UDP but with a custom application-layer protocol

for frame-based delivery. By contrast, **Gemini** and **Grok** conduct A/B testing on their protocols: **Gemini** alternates between QUIC/HTTP-3 (H3) and TCP/HTTP-2 (H2), while **Grok** switches between UDP/RTP and TCP/H2. The UDP/RTP and QUIC/H3 variants utilize frame-based transmission, whereas the TCP/H2 variant employs chunk-based delivery, which encodes multiple frames into a chunk. We observe that chunk-based transmission is poorly suited for live video streaming in human-GenAI interactions, as it introduces substantial additional delay (e.g., up to ~300 ms). Furthermore, it significantly increases bandwidth consumption, which makes chunk-based approaches more susceptible to performance degradation under network constraints, affecting VLM inference accuracy.

Finding 4: High Video Quality Helps, but High Frame Rates Offer Limited Benefit. Among applications that use frame-based transmission, we observe substantial variation in frame rate and in video quality. Our measurements indicate that higher-quality frames generally improve VLM inference accuracy, whereas high frame rates often provide limited benefit, likely because VLMs internally process video at relatively low effective rates. For example, **Yuanbao** has the lowest frame rate at 10 FPS (frames per second) but the largest average frame size (>11 KB), suggesting higher encoded video quality, and achieves about 60% accuracy on MVBench [34]. In contrast, the UDP/RTP variant of **Grok** has the highest frame rate (100 FPS) with much smaller frames (~2 KB, about one-fifth of **Yuanbao**), resulting in roughly 2× the throughput of **Yuanbao** but the lowest accuracy (below 40% for its free tier). To validate this observation, we further conduct controlled experiments by explicitly varying video resolution, H.264 quantization parameter (QP), and FPS over the measured applications, Gemini API [24], and a state-of-the-art VLM [68]. The results show that reducing video quality, either by lowering resolution or increasing QP, leads to 20–40% accuracy degradation across applications and models, whereas reducing frame rate from 30 FPS to 5 FPS has a limited impact.

Finding 5: Transport Protocols Critically Impact Robustness under Network Disruptions. The choice of transport protocol has a profound impact on application behavior under disrupted networks with limited bandwidth or packet loss. We observe that reliable protocols such as TCP retransmit data under network disruptions, which can significantly inflate conversational latency. The reliability mechanism also increases the risk of connection termination when acknowledgments are lost/delayed. For example, the TCP variant of **Grok** experiences conversational latency exceeding 60s, or even complete connection loss, under only 1% packet loss. The QUIC variant of **Gemini**, which is also built on reliable delivery, exhibits similar latency inflation under severe disruptions. In contrast, applications that use UDP avoid retransmissions and continue to function even under loss rates as high as 20%, without incurring dramatic increases in conversational latency.

Finding 6: VLM-Oriented Video Delivery Necessitates New Rate Adaptation Strategies. When available bandwidth is limited or packets are lost, all applications reduce their sending rate due to congestion control. Traditional adaptive bitrate (ABR) algorithms [42], widely used in human-facing video streaming, prioritize lowering resolution while maintaining a stable frame rate to preserve smooth playback. Our measurements show that this

approach does not translate well to video delivery for VLM consumption. For instance, when the loss rate increases from 3% to 5%, **ChatGPT**'s accuracy drops from 58% to 36%. It responds to packet loss with a traditional ABR-style strategy: maintaining a stable frame rate while reducing packet size, which likely indicates degraded video quality. In contrast, **Yuanbao** maintains video quality even at a 20% loss rate and instead reduces the frame rate from 10 FPS to 6–7 FPS. As a result, its accuracy remains unaffected even at a 20% loss. When available bandwidth is limited, all applications follow traditional ABR strategies by reducing video quality to cope with congestion, resulting in degraded inference accuracy.

Our measurements carry important implications for the future design of networked systems supporting real-time human–GenAI interaction, where long-standing assumptions from human-facing video streaming no longer hold. While our results reaffirm that frame-based streaming over unreliable transports such as UDP remains well-suited for delivering live video to VLMs, they also reveal the need to rethink traditional adaptation strategies. In particular, the human-centric practice of prioritizing higher frame rates over better quality is often counterproductive: VLMs benefit more from richer visual detail, even if it requires operating at lower frame rates. These findings call for a shift toward VLM-aware video delivery that balances networking behavior, video characteristics, and model inference.

Ethics. This work does not raise any ethical concerns. The user study conducted in this work was reviewed and approved by the Institutional Review Board (IRB).

2 Background

Vision Language Models (VLMs) are a class of multimodal foundation models that jointly process visual and textual inputs to perform tasks such as object recognition [72], scene understanding [2], and visual question answering [34]. Unlike traditional computer vision pipelines that operate on fixed tasks with specialized architectures, VLMs offer flexible, open-ended interfaces that let users express their goals in natural language. This versatility underpins emerging video-based human–GenAI interaction, in which users engage with generative AI agents through the cameras on their mobile devices. For example, users may ask an AI assistant to identify objects in view, describe a scene, or read text from signage, combining real-time vision processing with language understanding.

Video-Based Human–GenAI Interactions. Figure 1 illustrates the workflow of real-time video-based conversational interaction between humans and GenAI. At a high level, the user's mobile device (e.g., smartphone) continuously captures live camera frames and records an accompanying audio prompt (e.g., asking a question about the currently captured scene). Both the video and audio streams are transmitted over the Internet to the provider's backend, where a VLM performs inference and generates a response (often as synthesized speech) that is then returned to the user, acting as a conversational agent.

3 Measurement Methodology

We thoroughly measure five leading applications: **ChatGPT** [48] from OpenAI, **Doubao** [4] from ByteDance, **Gemini** [23] from Google, **Grok** [67] from xAI, and **Yuanbao** [56] from Tencent. To the best of

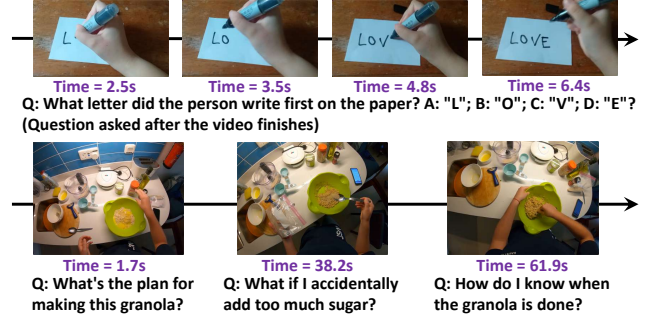


Figure 2: Two types of video-based human–GenAI interaction in our measurement study. Multiple-choice QA (top): The user provides an audio prompt containing a question and multiple-choice answer options about the completed video. **Chat (bottom):** The user interacts with GenAI in real time during video playback by issuing audio prompts.

our knowledge, these applications are the only ones that, at the time of our study, support video-based conversation between humans and GenAI on smartphones and are released by *Big Tech* [62]. To further validate our observations, we also conduct experiments using the Google Gemini API for video understanding [24] and StreamingVLM [68], a state-of-the-art VLM designed for streaming video input. We present the details for this experiment in §4.1.

Among these applications, **ChatGPT** only offers video conversational services in its paid tier, while **Gemini** and **Grok** provide them in both free and paid tiers. **Doubao** and **Yuanbao** are free and have no paid versions. To ensure a fair comparison across applications, we separately evaluate the free and paid versions of **Gemini** and **Grok**, treating them as distinct application variants. Furthermore, during our study, we observe that **Gemini** and **Grok** appear to perform A/B testing on their underlying network protocols. This is evidenced by the fact that, under otherwise similar experimental settings, these two applications use different protocol choices across users, locations, and repeated sessions, without a deterministic pattern observable from our measurements. Specifically, **Gemini** switches between TCP/HTTP2 (H2) and QUIC/HTTP3 (H3), a behavior also observed by a prior study on audio-only human–GenAI interaction conducted in 2025 [10]. In addition, **Grok** alternates between UDP/RTP and TCP/H2 as its transport/application-layer protocols. We therefore treat these protocol configurations as distinct application variants.

Thus, in total, our measurement study considers 11 different application configurations (four for **Gemini** and four for **Grok**, plus the remaining three applications). We conduct the same number of experiments for each of these 11 variants to enable fair comparison. When presenting results for **Gemini** and **Grok**, we append -F/-P (free/paid) and -Q/-T/-U (QUIC/TCP/UDP), respectively, *only* when their measurement results show distinguishable patterns.

To comprehensively measure these applications, we conduct experiments on two well-established datasets designed for evaluating VLMs: MVBench [34] and VideoLLM-online [7]. MVBench adopts a multiple-choice QA interaction mode, as illustrated in Figure 2. It designs a diverse set of tasks that probe video understanding from

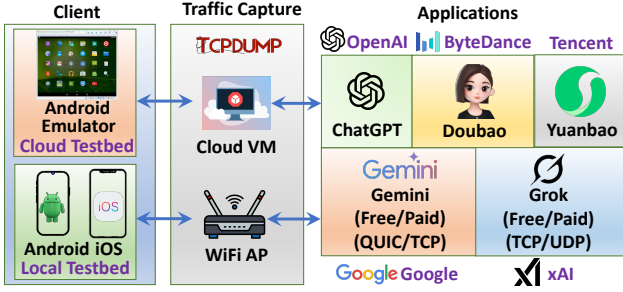


Figure 3: Measurement setup for five applications.

different perspectives (e.g., temporal reasoning, object tracking, and logical inference). These tasks evaluate the video understanding capability of the underlying VLMs, which form the foundation for effective communication with human users. Moreover, VideoLLM-online adopts a chat-style interaction mode, as demonstrated in Figure 2. It leverages egocentric videos from the Ego4D [25] dataset and designs prompts that enable users to interact with GenAI in real time (e.g., asking for help with the ongoing task). This setting more closely mimics real-world video-based interaction between humans and GenAI in everyday use.

Figure 3 presents our measurement setup. To facilitate experiments across different geolocations, we design an automated testbed running on cloud virtual machines (VMs). Specifically, for the mobile client, we run the *Genymotion* Android emulator (Android 14, arm64) [20]. To feed audio and video into the emulator, we provision a second Linux VM on the same subnet as the Genymotion instance, with the round-trip time (RTT) between them below 10 ms. On this Linux VM, we use *v4l2loopback-dkms* to create a virtual camera, *PulseAudio* to generate a virtual microphone, and *ffmpeg* to feed test audio and video into these virtual devices. We then use *MediaMTX* to stream the resulting audio and video to Genymotion via the RTSP protocol. For the VMs, we use *t3.2xlarge* instances on AWS, which provide sufficient resources to support smooth operation during experiments. We capture raw network traffic on the Linux VM hosting the Android emulator using *tcpdump*.

To validate the results collected from the cloud testbed, we also build a local testbed using a variety of Android and iOS smartphones, covering Android versions 12–16 and iOS versions 16–18. We connect the client device to a WiFi AP and capture the network traffic at the AP using *tcpdump*. With both the cloud and local testbeds, we are able to conduct large-scale experiments across the world. In particular, we conducted experiments during two periods, from 08/2025 to 11/2025 and from 03/2026 to 05/2026, covering nine locations across the U.S., three locations in mainland China, and Hong Kong SAR. Across these two measurement periods, we do not observe qualitative changes in our main findings. Across our measurement study, we do not observe substantial differences attributable to client type or location, and thus we report aggregated results unless otherwise noted.

We conduct experiments both in well-provisioned network environments (§4.1) and under disrupted network conditions with injected packet loss (§4.2) or constrained bandwidth (§4.3). Our experiments cover 13 controlled network conditions. In addition,

Dataset	Mode	# of Sessions	Dur. (hrs)
MVBench [34]	Q/A	20,350	213
VideoLLM-online [7]	Chat	27,400	817
Total	-	47,750	1,030

Table 1: Summary of the datasets and corresponding experiments conducted in our measurement study.

we replay cellular network traces released by Akamai [22], which capture time-varying packet loss and available bandwidth in real mobile environments. To comprehensively evaluate all 11 application configurations across these conditions, we select 50 videos from MVBench [34] (each 20–60s) and 5 videos from VideoLLM-online [7] (each 60–180s). All selected videos are 1080p (1920×1080), 30 FPS, and encoded with H.264 using quantization parameter (QP) values between 18 and 23. Unless otherwise specified, we use the original video quality in our experiments. We ensure that every application configuration is evaluated with identical repetition counts across all network conditions. In total, our dataset consists of 47,750 sessions spanning 1,030 hours of human–GenAI interaction, as summarized in Table 1. We comprehensively measure and analyze the following performance metrics from networking, VLM, and user-experience perspectives.

- *Throughput*. We measure the uplink throughput (i.e., from user to VLM backend) of video-based conversational GenAI services, which carries video traffic and is more bandwidth-intensive [6] than audio/text responses in the downlink. We compute throughput in one-second bins during the active user-input period, when the client continuously transmits uplink media data. For each one-second bin, we sum the transmitted uplink bits and report the resulting rate in Mbps. Throughput is jointly influenced by the inter-arrival time between consecutive frames/chunks and packet size, which we introduce next.

- *Frame/Chunk Inter-arrival Time* refers to the time interval between the sending time of consecutive frames or chunks. This metric reflects the media delivery pattern and contributes to transmission delay, thereby affecting end-to-end latency. Typically, there are two ways to stream media content: frame-based transmission, which delivers content frame-by-frame, and chunk-based transmission, in which each chunk contains multiple frames. We can distinguish these two transmission patterns by examining the inter-arrival time between consecutive packets that do not belong to the same media frame/chunk [10]. In frame-based transmission, inter-frame times are typically short, stable, and correlated with the sending frame rate (e.g., ~16 ms for 60 FPS and ~33 ms for 30 FPS). Note that the sending frame rate does not necessarily equal the source-video frame rate, as an application may upsample, downsample, or repeatedly transmit frames depending on its system design. In contrast, chunk-based transmission typically exhibits longer and more variable inter-arrival times, because packets are delivered in bursts separated by larger gaps at the chunk level.

As a video frame/chunk can be large, it is often segmented into multiple packets that are transmitted back-to-back. Thus, raw packet inter-arrival times contain both intra- and inter-frame/chunk gaps. For example, consider a frame-based stream at 100 FPS with an inter-frame interval of about 10 ms. If each frame is segmented into

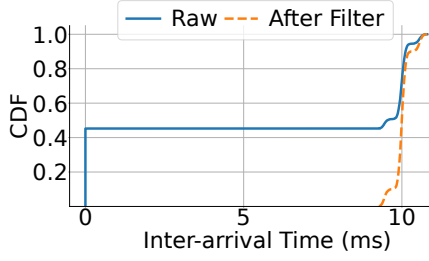


Figure 4: Packet inter-arrival time distribution for Grok using UDP before (raw) and after filtering values smaller than 0.5 ms.

three packets, then the gaps between packets 1–3 should be close to 0 because they belong to the same frame and are transmitted back-to-back, while the gap between packets 3 and 4 is about 10 ms because packet 4 belongs to the next frame. The same logic applies to chunk-based transmission: packets within the same chunk are transmitted back-to-back, while gaps between chunks reflect the chunk-level inter-arrival time.

In this paper, we use 0.5 ms as the threshold to filter out intra-frame/chunk packet gaps when computing frame/chunk inter-arrival time. Figure 4 illustrates this filtering using Grok-U as an example. The raw trace contains a large mass of near-zero inter-arrival times, corresponding to back-to-back packets within the same frame/chunk. After filtering, the distribution concentrates around 10 ms, indicating that Grok-U streams video at approximately 100 FPS. We also verify that nearby thresholds, such as 1 ms, produce similar inter-arrival-time estimates and do not change our conclusions.

- **Packet Size.** As discussed above, video frames are generally large and often exceed the maximum transmission unit (MTU) of each packet (e.g., 1,500 bytes [1]), requiring multiple packets to deliver a single frame. Thus, under ample bandwidth, the size of most packets in both frame-based and chunk-based transmission tends to be close to the MTU. However, packet size becomes particularly useful for analyzing transmission behavior under network disruptions. For instance, a widely used strategy to cope with dynamic network conditions is adaptive bitrate streaming [42], which lowers the video quality (e.g., by reducing resolution or increasing the compression level) while keeping the frame rate to maintain playback smoothness when available bandwidth is limited. In such cases, if the video quality becomes low enough that a frame can be carried in a single packet, we can observe a reduction in average packet size compared to scenarios with ample bandwidth.

- **Conversational Latency** is the time from when the user finishes an audio prompt to when GenAI begins its audio response. This metric captures user-perceived end-to-end latency, including network transmission, server-side processing (e.g., automatic speech recognition, ASR [32]), and VLM inference. Compared to audio-only interaction [10], video-based services require the VLM to process both the video input and the audio prompt, which is more computationally intensive than handling only the audio prompt [7]. Nonetheless, from the user’s perspective, the interaction still resembles an audio call. Thus, to provide a smooth experience, conversational latency should ideally be below one second [10].

- **End-to-End Task Accuracy.** To quantitatively assess the quality of GenAI responses, we compute end-to-end task accuracy on the tasks from the MVBench dataset [34]. As introduced earlier, MVBench provides multiple-choice QA tasks designed to evaluate video understanding capabilities. For each task, the audio prompt contains the question and all candidate answer options. We treat accuracy as a binary metric: a response is counted as correct only if it matches the ground-truth answer option; otherwise, it is counted as incorrect. Characterizing the task accuracy of the investigated applications offers insight into user experience, as it reflects whether the system can correctly interpret and respond to user queries. To reduce variability introduced by human speech, we generate all audio prompts using a standardized voice synthesized by the text-to-speech (TTS) [51] model from ElevenLabs [19]. This allows the audio-processing components (e.g., ASR [71]) to operate more reliably, such that the measured accuracy primarily reflects the system’s video understanding capability (i.e., VLM inference accuracy).

- **Mean Opinion Score (MOS).** Collecting mean opinion scores (i.e., user ratings) is a widely used approach for evaluating quality of experience (QoE) in multimedia systems [30]. We therefore conduct an IRB-approved user study with 37 participants (15 female, 22 male; average age: 29.3 ± 5.1). Experiments are run on the local testbed, and participants observe the interactions in person. We employ the VideoLLM-online dataset, whose chat-style interaction closely mimics real-world video-based human–GenAI interaction scenarios. Participants are instructed to imagine they are personally using the application and conversing with the GenAI system, and to rate their overall experience after each experiment on a five-point scale (1: very bad; 5: very good). To avoid bias, the application configurations and network conditions are randomized for each experiment. Moreover, we do not prescribe any rating weights to avoid biasing participants’ judgments of their overall experience. Instead, we conduct post-experiment interviews to understand the rationale behind participants’ ratings, following common practice in the HCI community [8, 9].

4 Measurement Results

4.1 Well-Provisioned Networks

We first evaluate the performance of these applications under unconstrained network conditions.

Network Infrastructure. We begin by examining the network infrastructure supporting these applications from two aspects: (1) transport and application-layer protocols, and (2) characteristics of the entry server.

(1) **Protocols.** As shown in Table 2, we observe three distinct protocol choices across these platforms. ChatGPT and Yuanbao use UDP/RTP, which is widely adopted in live video streaming and typically employs frame-based transmission for media content [6, 15]. Doubao also uses UDP at the transport layer, but it does not rely on a standard protocol at the application layer (likely a customized RTP-like protocol). As examined in §3, Grok and Gemini appear to conduct A/B testing on their networking protocols: Grok switches between UDP/RTP and TCP/H2, while Gemini alternates between QUIC/H3 and TCP/H2. These protocol choices have a profound impact on

Application	Protocols	RTT (ms)	Frame Size (KB)
ChatGPT	UDP/RTP	21.3/8.4	7.0/1.5
Doubao	UDP/-	17.4/3.9	2.5/0.3
Gemini	QUIC/H3 TCP/H2	19.1/5.5	1.4/0.1 -
Grok	UDP/RTP TCP/H2	13.7/2.9	2.0/0.1 -
Yuanbao	UDP/RTP	20.1/5.8	11/2.6

Table 2: Summary of (1) the transport/application layer protocols for delivering video data. H2 and H3 stand for HTTP/2 and HTTP/3, respectively. Doubao does not rely on a standard protocol at the application layer, (2) the round-trip time (RTT) measured between the client and entry server from different client locations in the U.S. and Asia, shown as average/standard deviation in milliseconds, and (3) the frame size for five applications employing frame-based transmission, shown as average/standard deviation in kilobytes. Applications using TCP/H2 employ chunk-based transmission, making the frame size not directly calculable.

transmission behavior, especially under disrupted networks (§4.2 and §4.3).

(2) *Entry server.* We next examine the entry server of each application. It refers to the ingress/egress point between the client and the provider’s infrastructure, which typically acts as a relay between the user device and the data center hosting backend services such as VLM inference [11, 17]. We measure the RTT between the client and its assigned entry server using ICMP pings. With clients in different locations, we also confirm that none of the evaluated applications use *anycast* routing [37, 44], following the approach adopted in prior work [16]. This allows us to infer the geolocation of entry servers from their IP addresses. We use *MaxMind* [43] and *ipinfo.io* [27] for geolocation and provider lookup, and *whois* [61] for additional ownership lookup. These tools yield consistent results across all observed entry servers for location and owner.

Table 2 summarizes the RTT to the entry servers across all of our client locations (§3). All applications can allocate nearby entry servers to users around the world where they are allowed to operate, with average RTTs typically below or around 20 ms. This is a positive sign and represents a significant improvement compared to prior measurement studies on other real-time communication applications such as social virtual reality [16], video conferencing [15], and audio-based human–GenAI interaction [10]. Having nearby entry servers is important because the path between the client and entry server traverses the public Internet, for which the network conditions may not be stable. Low RTT helps quickly adapt to network dynamics via congestion control [5] and rate control [42], both of which are sensitive to RTT [17].

An earlier study on audio-based human–GenAI calling conducted by Cheng *et al.* in early 2025 [10] also reported the network infrastructure of **ChatGPT** and **Gemini**. We compare our observations with those of that study to examine how the infrastructure has evolved. The prior study reported that **ChatGPT** and **Gemini** did not always assign users to nearby entry servers, especially for users

located in the central U.S. In contrast, our measurements show that both applications now more consistently route users to nearby entry servers, indicating rapid evolution and optimization of the network infrastructure for human–GenAI interaction. In terms of infrastructure ownership, the entry servers used by **ChatGPT** in our measurements are registered to Microsoft, whereas the prior study reported Oracle-hosted entry servers [10]. This shift may be related to the recently announced expansion of the OpenAI–Microsoft partnership, under which OpenAI committed to purchasing additional Azure services [49]. **Gemini** continues to use Google’s own infrastructure. For the remaining applications not covered by Cheng *et al.* [10], **Doubao** uses ByteDance’s own infrastructure when clients are in mainland China and Hong Kong, and relies on *Datacamp Limited* when clients are in the U.S. **Yuanbao** uses Tencent’s own infrastructure, whereas **Grok** consistently relies on Cloudflare.

Uplink Transmission Analysis. We next analyze uplink throughput and frame/chunk inter-arrival time. The uplink carries both video input and audio prompts from users. We first separate video packets from other uplink traffic, leveraging the fact that video streams consume significantly more bandwidth. For applications using RTP at the application layer, we exploit the SSRC field to distinguish audio and video sessions and identify the session with a much higher bandwidth as video. For other applications, we filter out packets smaller than 400 bytes, following prior work on audio-based GenAI calling [10], which showed that audio packets are typically below this threshold.

(1) *Uplink behavior during active video capture.* Figures 5(a) and (b) show the uplink throughput and frame/chunk inter-arrival time when the user is continuously feeding new visual content (*e.g.*, by actively moving the camera). Each boxplot reports the 5th, 25th, 50th, 75th, and 95th percentiles, with blue dots indicating the mean. We observe a large variance in consumed bandwidth across applications, ranging from an average of ~ 0.8 Mbps for **Gemini-Q** to ~ 3 Mbps for **Grok-T**. Moreover, the inter-arrival times reveal two distinct transmission strategies (§3). **ChatGPT**, **Doubao**, **Gemini-Q**, **Grok-U**, and **Yuanbao** exhibit small and relatively stable inter-arrival times, indicating *frame-based transmission*. From these values, we can infer the effective frame rate: 30 FPS (33 ms per frame) for **ChatGPT**, 60 FPS (16.7 ms per frame) for **Doubao** and **Gemini-Q**, 100 FPS (10 ms per frame) for **Grok-U**, and 10 FPS (100 ms per frame) for **Yuanbao**. Combining throughput and frame rate, we estimate the average frame sizes shown in Table 2, which correlate with the effective video quality. **Yuanbao** has the largest average frame size (over 10 KB), whereas **Doubao**, **Gemini-Q**, and **Grok-U** have smaller frames (no more than 2.5 KB), suggesting that **Yuanbao** likely streams at a higher quality than the other three. In contrast, **Gemini-T** and **Grok-T** show high inter-arrival times, indicating *chunk-based transmission* where multiple frames are batched into a chunk.

We now discuss how these design choices affect throughput.

(i) Comparing **Gemini-T** with **Gemini-Q** (1.7 vs. 0.7 Mbps) and **Grok-T** with **Grok-U** (2.9 vs. 1.7 Mbps), we observe that chunk-based transmission consumes substantially more bandwidth than frame-based. Since these variants belong to the same applications, it is reasonable to assume that the camera capture settings (frame rate

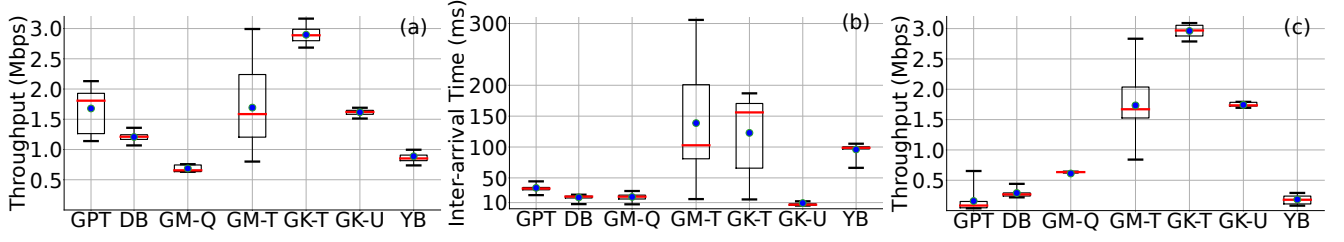


Figure 5: Performance of the evaluated applications under unconstrained network conditions. GPT: ChatGPT; DB: Doubao; GM: Gemini; GK: Grok; YB: Yuanbao. -Q/-T/-U denote QUIC/TCP/UDP, respectively. (a) Uplink throughput and (b) frame/chunk inter-arrival time for dynamic video content; (c) Uplink throughput for static video content.

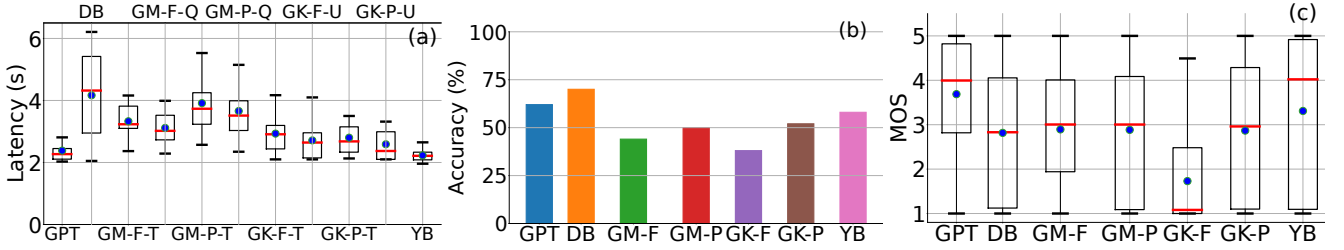


Figure 6: Performance of the evaluated applications under unconstrained network conditions. -F/-P denote free/paid tiers. (a) Conversational latency; (b) Accuracy in multiple-choice QA tasks; and (c) Mean Opinion Score (MOS) from the user study.

and video quality) are similar. Thus, the higher bandwidth consumption largely stems from the transport behavior: batching multiple frames into bursts leads to longer inter-arrival times between chunks and thus higher instantaneous throughput. Such bursty transmission is common in on-demand video streaming, where pre-encoded content is delivered in chunks to efficiently utilize bandwidth. Nonetheless, for video-based human-GenAI interaction, the uplink carries live-captured content, fitting well into RTC [45]. In this scenario, low and stable bandwidth usage is preferable, as it reduces the frequency of rate adaptation when available bandwidth is limited, thereby avoiding quality switches that degrade QoE [40].

(ii) For frame-based transmission, frame rate and video quality jointly determine the throughput. For example, **Grok-U** streams at 100 FPS; even though its average frame size is only about 2 KB (the second smallest in Table 2), its throughput still exceeds 1.5 Mbps. Also, streaming high-quality content increases throughput due to larger frame sizes. For instance, **Yuanbao** has the largest frame size among frame-based applications (Table 2), and thus, despite its low frame rate (10 FPS), its throughput still approaches 1 Mbps. However, high frame rate and high quality do not both translate into improved VLM inference accuracy. As shown in Figure 6(b), **Yuanbao** achieves 58% accuracy, whereas **Grok-F** reaches only 38% accuracy. We later conduct controlled experiments that explicitly vary the quality and frame rate of input videos, showing that higher streaming video quality tends to improve inference accuracy, whereas simply increasing frame rate does not yield similar benefits.

(2) *Uplink behavior when feeding static video content.* We next examine the uplink bandwidth usage of these applications when the captured scene is static. Widely used video codecs, such as H.264, can exploit inter-frame compression: a later frame (e.g., a P-frame)

can be encoded by referencing an earlier frame (e.g., an I-frame), rather than transmitting the full visual content again. If such inter-frame compression is enabled, it can substantially reduce bandwidth usage when streaming a static scene. To assess whether applications implement such optimizations, we conduct experiments in which the camera is held still, pointed at an unchanging scene. The resulting throughput for these applications is shown in Figure 5(c). Compared to their results during active video capture in Figure 5(a), **ChatGPT** (1.68→0.04 Mbps), **Doubao** (1.20→0.22 Mbps), and **Yuanbao** (0.89→0.12 Mbps) significantly reduce uplink throughput when the scene is static. In contrast, both **Gemini** and **Grok**, regardless of the underlying protocol, continue to send video at similar rates as in the active phase. This indicates that they do not reduce bandwidth usage when streaming static scenes. However, this does not necessarily imply a failed optimization. Instead, it may also reflect a deliberate design choice. One possible explanation is codec behavior: the video encoder may treat frames more independently, rather than exploiting inter-frame compression. This can help maintain a smooth sending rate, which can also be a desirable design choice in streaming systems [55].

Conversational Latency and End-to-End Task Accuracy. Figure 6(a) reports the conversational latency across the 11 application variants. First, we observe that all applications exhibit conversational latency greater than 2s, exceeding the 1s threshold generally considered necessary for smooth conversation [10]. Among them, **ChatGPT** and **Yuanbao** achieve the lowest latency (~2s), while **Doubao** has the highest, reaching up to 6s. Interestingly, as shown in Figure 6(b), these three applications also yield higher accuracy than others, roughly at or above 60%, with **Doubao** achieving ~70%.

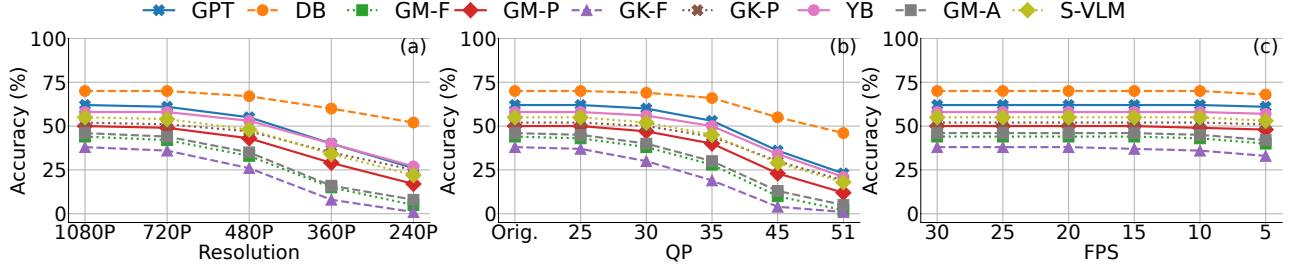


Figure 7: Controlled experiments on the impact of (a) video resolution, (b) quantization parameter (QP) of the H.264 codec, and (c) FPS on end-to-end task accuracy. We evaluate the same seven app variants shown in Figures 6(b) and 6(c), as well as Gemini API [24] (GM-A) and StreamingVLM (S-VLM) [68].

In general, higher inference accuracy implies more sophisticated models with larger parameter counts and thus higher computation overhead, which would normally increase latency. One potential explanation for **ChatGPT** and **Yuanbao** achieving relatively low latency while maintaining high accuracy is that they stream high-quality video, as suggested by their larger frame sizes in Table 2. Higher-quality visual input is known to improve VLM performance [33], which may allow these applications to rely on less complex models while still attaining competitive accuracy. In contrast, **Doubao** uses relatively low-quality video input and likely relies more heavily on model complexity to achieve high accuracy, which contributes to its longer latency. We also note that while **Yuanbao** streams at only 10 FPS, this does not appear to hurt its accuracy, which may indicate that VLMs often process video at low frame rates and do not require high-FPS input.

For **Gemini** and **Grok**, the frame-based variants (QUIC/UDP) generally show 100–300 ms lower latency than their chunk-based TCP variants, thanks to their smaller inter-arrival times observed in Figure 5(b). This highlights the benefit of frame-based transmission for live streaming. Also, the paid versions generally achieve higher accuracy than the free ones (e.g., 52% for **Grok-P** vs 38% for **Grok-F**), which aligns with the expectation that providers allocate more powerful models or better compute resources to paid users. However, this can also increase inference overhead. For example, **Gemini-P** has higher accuracy than **Gemini-F** but also higher latency, suggesting that the paid tier may rely on a heavier model without proportionally more GPU resources. In contrast, **Grok-P** is both more accurate and faster than **Grok-F**, indicating that **Grok** may allocate more compute resources to the paid tier.

It should be noted that the accuracy of these applications (up to ~70%) is still well below ideal levels. As our user study later reveals, participants frequently mention that these systems fail to reliably interpret the visual references provided in the video. Improving accuracy by training more powerful models will likely increase computational demands and place additional stress on the underlying infrastructure, motivating the need for better system and network support.

Controlled Experiments on the Impact of Video Quality and Frame Rate on Task Accuracy. To directly validate the impact of video quality and frame rate on end-to-end task accuracy, we conduct controlled experiments using the original MVBench videos, which are 1080p, 30 FPS, and encoded with H.264 using QP values

between 18 and 23 (§3). Starting from this baseline, we vary one factor at a time: video resolution from 1080p to 240p, H.264 QP from the original value to 51 (where a larger QP indicates stronger compression and lower encoded video quality), and frame rate from 30 FPS to 5 FPS, while keeping the remaining factors unchanged. In addition to the measured applications, we also evaluate Gemini API [24] and StreamingVLM [68], a state-of-the-art VLM designed for streaming video input, to examine whether the trend generalizes across different VLM models.

The results are shown in Figure 7. Overall, the evaluated applications and models show two clear trends: reducing encoded video quality substantially degrades accuracy, whereas reducing frame rate has a much smaller impact. For video quality, accuracy drops consistently when resolution decreases from 1080p to 240p (Figure 7(a)) or when QP increases to 51 (Figure 7(b)), leading to 20–40% accuracy degradation across applications and models. In contrast, reducing frame rate from 30 FPS to 5 FPS has little impact (<5%) for these applications and models. These results reveal that, for VLM inference, video quality is a dominant factor for accuracy, while high frame rate may provide limited additional benefit. A concurrent study also observes similar patterns on a self-built benchmark, further corroborating our findings [64].

We note that these results are based on the full set of tasks and video content evaluated in our study. We do not separately analyze how different task types or video characteristics affect the performance of the underlying VLMs used by these applications, although these factors may influence the observed trends. For example, tasks involving fast-moving objects or temporal reasoning may benefit more from higher frame rates than the aggregate results suggest. A systematic evaluation across different task categories and video characteristics is an important direction for future work and is further discussed in §5.

User Experience. We now present findings from the user study. From post-experiment interviews, participants most frequently cited “whether the app correctly understands my query” as the primary factor affecting their experience, followed by response delay. This aligns with our quantitative metrics for accuracy and conversational latency. These perceptions are reflected in the MOS results in Figure 6(c). We observe substantial variability in user ratings: all applications receive a MOS of 1 at the 5th percentile, and all except **Grok-F** achieve a MOS of 5 at the 95th percentile. Our post-experiment interviews reveal that users tend to assign

App.	CPU (%)	Mem. (%)	Battery	
			Level (%)	Temp. (°C)
Doubao	13.2/2.4	4.9/1.1	91/1.2	38/1.4
ChatGPT	15.2/2.8	3.2/0.7	89/2.5	40/1.8
Gemini	13.7/2.4	3.5/0.9	93/1.8	37/1.2
Grok	7.3/0.9	2.8/0.7	93/1.5	38/1.0
Yuanbao	18/3.3	3.7/1.2	88/2.9	41/2.4

Table 3: Summary of (1) CPU usage, (2) memory usage, and (3) battery level and temperature after 30 minutes of continuous use. Each cell reports mean/standard deviation. The battery level is 100% and the temperature is ~25 °C at the beginning of each experiment.

a rating of 1 when the application fails to accurately understand their query. This aligns with results in Figure 6(b), which shows that all applications achieve accuracy below 75%. Among these applications, **ChatGPT** (mean: 3.7, std: 1.3) and **Yuanbao** (mean: 3.3, std: 1.7) obtain the highest MOS, benefiting from their relatively low latency and high accuracy, as shown in Figures 6(a) and 6(b). In contrast, **Grok-F** receives the lowest MOS (mean: 1.7, std: 1.1) due to its poor accuracy. Although **Doubao** achieves the highest accuracy among all applications, its long latency degrades QoE, resulting in an average MOS of only 2.8 (std: 1.5). Overall, none of the applications attains an average MOS above 4, reflecting the combined effect of suboptimal accuracy and high conversational latency in current applications.

Resource Utilization and Energy Consumption. We conduct experiments to characterize the resource utilization and energy consumption of the applications under study using *SoloX* [54]. We use a OnePlus 10 Pro smartphone as the client device. For each experiment, we charge the battery to 100% and wait until it cools down (~25 °C), then run a given application continuously for 30 minutes. Each application is tested five times. The results are summarized in Table 3. Overall, these applications exhibit reasonable CPU and memory footprints, averaging below 20% CPU utilization and 5% memory usage. Among them, **Yuanbao** and **ChatGPT** consume slightly more resources, with CPU utilization exceeding 15%, battery level dropping below 90%, and temperature rising above 40 °C. This is likely because they capture and stream higher-quality video than others (Table 2).

Summary and Insights. Our measurement study under unconstrained networks reveals key insights for building high-QoE networking support for video-based human-GenAI interaction. We observe that two factors largely determine user experience: end-to-end task accuracy and conversational latency. These factors embody trade-offs between computation and communication. From a networking perspective, our results suggest that GenAI services should: (1) prefer frame-based transmission for live video input to reduce transmission latency; and (2) deliver high-quality video to improve VLM inference accuracy, while avoiding unnecessarily high frame rates that increase bandwidth demand without improving accuracy.

4.2 Lossy Networks

We next evaluate the performance of these applications under disrupted networks. We begin by studying how they react to packet loss, which is particularly prevalent in mobile networks [21]. For

all experiments in this subsection, we maintain sufficient available bandwidth to isolate the impact of packet loss. Following prior work on RTC [16], we inject controlled packet loss on both uplink and downlink paths with rates of {1, 3, 5, 7, 10, 20}%. We examine how packet loss affects each application’s transmission behavior, conversational latency, end-to-end task accuracy, and MOS, as shown in Figure 8. To further evaluate performance under realistic mobile-network loss patterns, we replay cellular loss traces released by Akamai [22] (avg: 10.1%; std: 21.0%; p50: 0%; p90: 56%). These traces exhibit *bursty* loss patterns, with instantaneous loss rates reaching up to 80%, a characteristic commonly observed in practice [12]. For each video, we randomly sample three trace segments that match the video’s duration and replay the same segments across all applications to ensure a fair comparison. The results are shown in Figure 9. For clarity, we aggregate the free and paid tiers for **Gemini** and **Grok** and differentiate their protocol variants to highlight the impact of protocol choice under disrupted networks. We have two main observations.

(1) *Impact of transport protocol on conversational latency and inference accuracy.* From Figure 8(a), we observe that uplink throughput decreases for all applications as the packet loss rate increases. This is expected, as modern congestion control algorithms [5, 26] widely treat packet loss as a congestion signal and reduce the sending rate accordingly. Among all applications, TCP-based variants (**Gemini-T** and **Grok-T**) react particularly sharply to packet loss. For example, under only 1% loss, the throughput of **Grok-T** drops from 2.9 Mbps to 1.7 Mbps on average (a 41% reduction), and **Gemini-T** drops from 1.7 Mbps to 0.5 Mbps (a 71% reduction). Moreover, at just 1% packet loss, both TCP-based variants frequently fail to operate.

Even when the TCP-based variants remain functional, their conversational latency grows dramatically with packet loss. For instance, at 1% loss, the average latency of **Grok-T** increases to 37s (with a maximum of 61s), as shown in Figure 8(d), while its accuracy drops to around 10%, as shown in Figure 8(e), yielding MOS values close to 1 (very poor), as indicated in Figure 8(f). Similarly, for **Gemini-T**, the conversational latency exceeds 6s at 5% loss and reaches roughly 13s at 10% loss (Figure 8(d)), accuracy falls below 30% beyond 7% loss (Figure 8(e)), and MOS drops below 1.5 for higher than 5% loss (Figure 8(f)). The cellular-trace results in Figure 9 show a similar trend. **Grok-T** has an average conversational latency of 67.8s, task accuracy of only 2%, and an average MOS close to 1. **Gemini-T** also degrades substantially, reaching 11.4s average latency and 12% accuracy. Our post-experiment interviews show that participants who assigned MOS scores of 1 to TCP-based variants most often attributed their ratings to “long response delays” or “connection drops.” This suggests that severe latency inflation and session failures are the primary factors driving the poor MOS for these variants.

One plausible explanation for these behaviors is the reliable-delivery design of TCP [18] and QUIC [29]. To ensure reliability, these protocols retransmit lost packets. This behavior is also observed in our measurements: for TCP-based applications, we observe substantial retransmissions after introducing packet loss, as indicated by traffic analysis tools such as Wireshark [63]. Such retransmissions can inflate latency. Moreover, if lost packets cannot be recovered within the application’s timeout window, the session

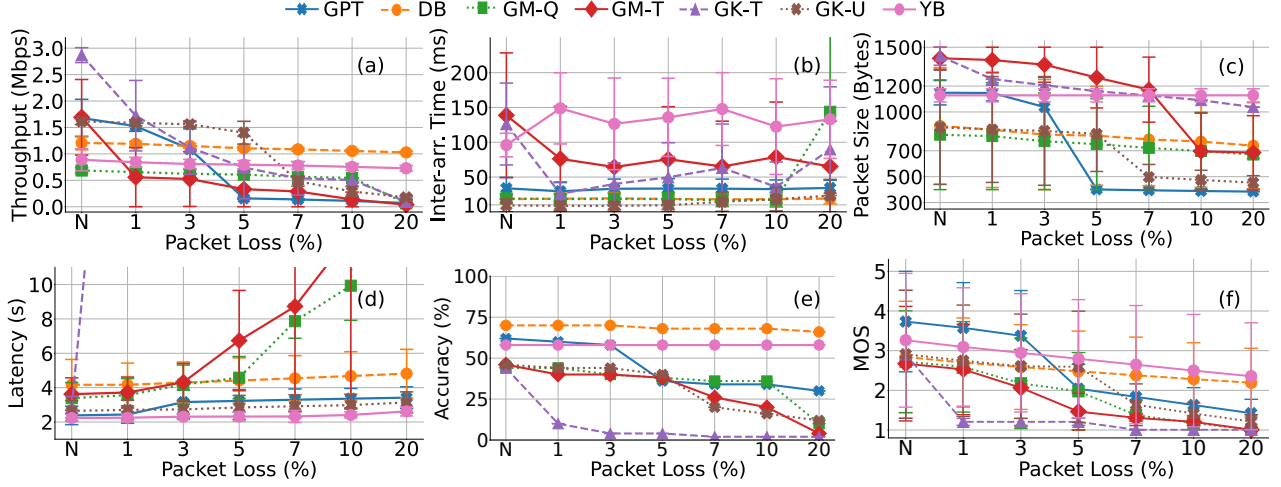


Figure 8: Performance of evaluated applications with controlled packet loss. N: No packet loss is applied. (a) Uplink throughput, (b) inter-arrival time, (c) packet size, (d) conversational latency (the average latency of Grok-T is >30s under packet loss), (e) end-to-end task accuracy, and (f) MOS.

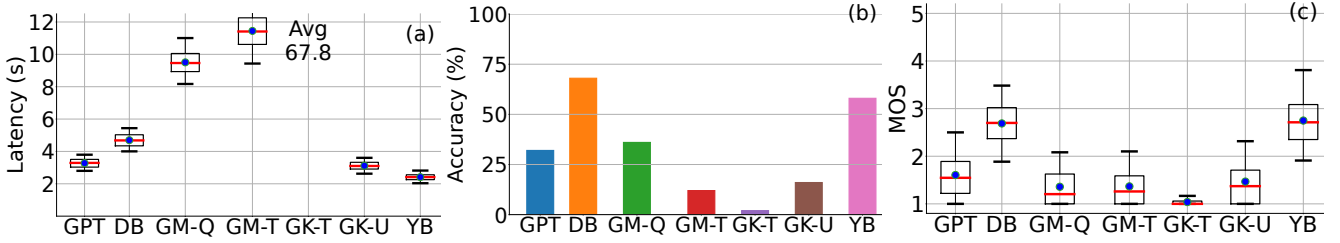


Figure 9: Performance of the evaluated applications when replaying the cellular loss traces released by Akamai [22]. (a) Conversational latency, (b) end-to-end task accuracy, and (c) MOS.

may terminate, which helps explain the high failure rates observed for TCP-based variants. As QUIC is also designed as a reliable transport protocol, it may suffer from similar issues in this scenario. This is reflected in **Gemini-Q**, whose latency begins to grow sharply beyond 7% loss (e.g., ~8s at 7% and 10s at 10%, as shown in Figure 8(d)), accompanied by a drop in MOS to below 1.5 (Figure 8(f)). Moreover, under cellular loss traces, **Gemini-Q** reaches 9.5s average latency and only 36% accuracy (Figure 9).

In contrast, applications using UDP, including **ChatGPT**, **Doubao**, **Grok-U**, and **Yuanbao**, show much more stable latency under packet loss. Their latency does not increase significantly as the packet loss rate grows (Figure 8(d)), and they also exhibit low latency variance under cellular loss traces (Figure 9(a)). This is because UDP does not enforce reliable, in-order delivery at the transport layer, allowing applications to avoid waiting for late or lost packets. This follows the common design practice in RTC systems to preserve conversational smoothness.

(2) *Different adaptation strategies and their impact on inference accuracy.* We now examine how UDP-based applications adapt video quality when packet loss reduces their sending rate. Applications typically rely on adaptive bitrate (ABR) mechanisms to adjust video properties so that frames can still be delivered under constrained bandwidth. In traditional live video streaming for human viewers,

ABR schemes usually sacrifice resolution before frame rate, since variations in frame rate tend to be more noticeable and disruptive than changes in resolution [42]. However, as we previously observed in §4.1, for VLM consumption, video quality is more important than frame rate.

This is reflected in the ABR strategies used by the UDP-based applications. Take **ChatGPT** and **Grok-U** as examples: their accuracy drops significantly under packet loss. As shown in Figure 8(e), **ChatGPT**'s accuracy decreases from 58% to 36% between 3% and 5% loss, and **Grok-U**'s accuracy decreases from 40% to 20% between 5% and 7% loss. Under cellular loss traces, **ChatGPT** achieves only 32% accuracy, while **Grok-U** achieves only 16% accuracy (Figure 9(b)). To understand the reason, we examine their adaptation behavior under packet loss (Figure 8). When packet loss exceeds certain thresholds, both applications exhibit sharp throughput reductions but largely unchanged inter-arrival times, indicating stable frame rates, along with substantial decreases in packet size. For example, as the loss rate increases from 3% to 5%, **ChatGPT**'s throughput drops from above 1 Mbps to about 0.3 Mbps (Figure 8(a)), while its inter-arrival time remains nearly constant (Figure 8(b)). At the same time, its average packet size decreases from over 1,000 bytes to below 500 bytes (Figure 8(c)). A similar pattern is observed for **Grok-U**: as the loss rate increases from 5% to 7%, throughput drops from around 1.5 Mbps to 0.5 Mbps, with stable inter-arrival times but packet

sizes shrinking from above 800 bytes to below 500 bytes. Since our controlled experiments confirm that reducing encoded video quality lowers task accuracy (§4.1), the observed degradation in accuracy with a stable frame rate and reduced packet size likely indicates that these applications reduce encoded video quality while maintaining FPS under packet loss. This behavior is consistent with traditional ABR strategies that prioritize frame rate.

In contrast, **Yuanbao** appears to prioritize preserving video quality. Figure 8(c) shows that its average packet size remains above 1,000 bytes even under 20% loss, while its inter-arrival time increases from 100 ms (10 FPS) to ~140–150 ms on average (6–7 FPS), with larger variance, as shown in Figure 8(b). This indicates that **Yuanbao** primarily drops frame rate to cope with reduced throughput while preserving video quality. Under this strategy, its accuracy remains largely stable even under a loss rate of up to 20% (Figure 8(e)).

We also observe that **Doubao** is less affected by packet loss: its throughput, frame rate, and packet size change only marginally as the loss rate increases, likely because the underlying congestion control is configured to be less sensitive to loss. This setting may reflect a deliberate design decision: given that **Doubao** achieves high inference accuracy under normal conditions (§4.1), it may tolerate a moderate amount of content loss without significantly degrading inference quality. Our empirical results are consistent with this hypothesis, as **Doubao**'s accuracy degrades only slightly even at a 20% packet loss rate (Figure 8(e)) and achieves the highest accuracy, 68%, under cellular loss traces (Figure 9(b)).

Summary and Insights. Our measurement study under packet loss reveals two key design insights for video-based human-GenAI interaction. First, UDP-based designs are better suited for maintaining low conversational latency under lossy conditions, whereas reliable transports such as TCP and QUIC can significantly inflate latency and even cause connection failures due to retransmission-based recovery. Second, when designing ABR strategies, systems should move away from the conventional practice of lowering quality to preserve frame rate and instead prioritize maintaining high video quality, which is far more critical for VLM inference accuracy.

4.3 Bandwidth-Constrained Networks

We finally evaluate these applications under constrained bandwidth. We throttle the bandwidth only for the uplink, which carries video traffic. For all experiments in this subsection, we do not inject packet loss to isolate the impact of available bandwidth. We first conduct controlled experiments by configuring the available uplink bandwidth to {3, 2, 1.5, 1, 0.7, 0.5} Mbps. This range covers the bandwidth consumed by these applications under unconstrained networks (0.7–2.9 Mbps on average, as shown in Figure 5(a)), ensuring that each application eventually experiences insufficient uplink bandwidth. The results are shown in Figure 10.

To further evaluate performance under mobile-network bandwidth dynamics, we replay the available-bandwidth time series from cellular traces released by Akamai [22] (avg: 8.4 Mbps; std: 10.1 Mbps; p25: 0.5 Mbps; p50: 2.9 Mbps). In these traces, although the average bandwidth is relatively high, the instantaneous bandwidth often drops to a level insufficient for these applications, consistent with throughput fluctuations observed in today's cellular

networks [21]. For each video, we randomly sample three trace segments that match the video duration from periods when the available bandwidth falls below 0.5 Mbps, and replay the same segments across all applications to ensure a fair comparison. The results are shown in Figure 11.

We highlight two main observations.

(1) *Inappropriate ABR strategies.* Following the analysis approach used in our packet-loss experiments (§4.2), we infer that UDP-based applications primarily react to constrained bandwidth by reducing encoded video quality. This is reflected in a common adaptation pattern: below certain bandwidth thresholds, they reduce packet size while maintaining or even increasing the frame rate, accompanied by noticeable drops in accuracy (Figure 10). This behavior is similar to how **ChatGPT** and **Grok-U** react to packet loss in §4.2. A particularly striking example is **Yuanbao**. Even at 3 Mbps, which is more than 3× its average throughput under unconstrained networks, **Yuanbao** reduces its average packet size from 1,128 bytes to 656 bytes, a 42% reduction, while *increasing* its frame rate from 10 FPS (100 ms inter-arrival time) to 100 FPS (10 ms inter-arrival time), as shown in Figure 10. This ABR behavior is essentially the opposite of its more effective strategy under packet loss (§4.2), in which **Yuanbao** preserves video quality while reducing the frame rate. As a result, its accuracy drops from 58% under normal conditions to 24%, and its average MOS decreases from 3.6 to 2.3, even when 3 Mbps of bandwidth remains available. A similar trend appears under cellular bandwidth traces, where **Yuanbao** achieves only 18% accuracy and an average MOS of 2.0 (Figure 11).

Moreover, we observe that TCP/QUIC-based variants behave particularly poorly under constrained bandwidth, similar to packet-loss scenarios. For example, at 3 Mbps, **Grok-T** is often non-functional, with conversational latency reaching up to 71s (Figure 10(d)), accuracy close to zero (Figure 10(e)), and MOS pinned at 1 (Figure 10(f)). **Gemini-Q** and **Gemini-T** also experience significant latency inflation as bandwidth decreases. For instance, under a 0.5 Mbps network, both exhibit average latency greater than 8s, as shown in Figure 10(d). These TCP/QUIC-based variants also exhibit degraded performance over cellular traces: **Grok-T** reaches 76.7s average latency with only 4% accuracy and MOS close to 1, while **Gemini-Q** and **Gemini-T** both reach ~7s average latency and an average MOS of ~2.0 (Figure 11). As in the packet-loss case (§4.2), the root causes are likely related to retransmissions and the requirement of in-order delivery for reliable transport protocols. This happens because the data to be sent is encoded at bitrates higher than what the network can sustain: packets accumulate in queues, RTT inflates, and eventually timeouts or duplicate ACKs trigger retransmissions. This also explains the large variance in inter-arrival times for **Gemini-T** and **Grok-T**, as shown in Figure 10(b).

(2) *Overly conservative bandwidth utilization.* Figure 10(a) also reveals that applications tend to utilize bandwidth conservatively. When the available bandwidth is 3 Mbps, applications such as **Grok-U** and **Yuanbao** already begin to reduce their sending rate, even though their normal operating rate under unconstrained networks is below 1.7 Mbps. When the bandwidth is limited to 2 Mbps, almost all applications send at or below 1 Mbps, with **Grok-U**, **Gemini-T**, and **Yuanbao** reducing their throughput to around 0.5 Mbps. This behavior is unlikely to be driven solely by congestion control, which

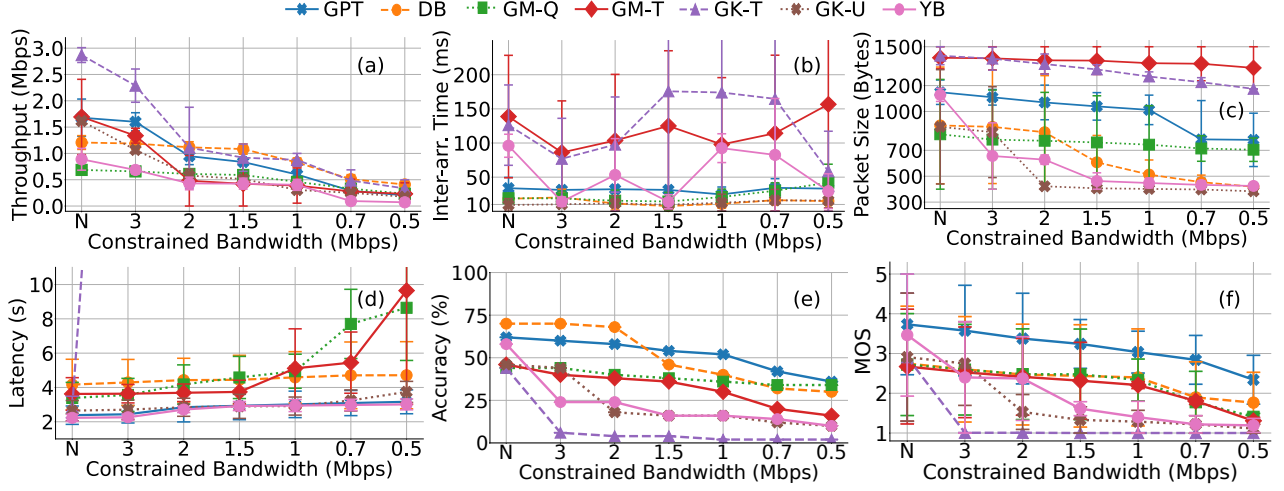


Figure 10: Performance of the evaluated applications under controlled uplink bandwidth constraints. N: No bandwidth throttling is applied. (a) Uplink throughput, (b) inter-arrival time, (c) packet size, (d) conversational latency (the average latency of Grok-T is >35s under constrained bandwidth), (e) end-to-end task accuracy, and (f) MOS.

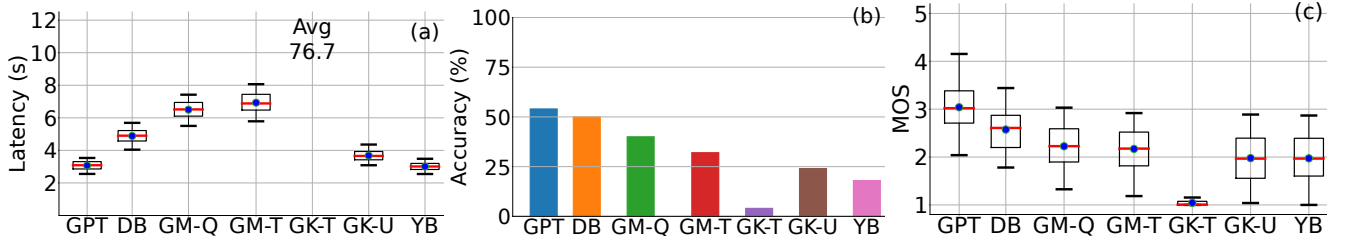


Figure 11: Performance of the evaluated applications when replaying the cellular bandwidth traces released by Akamai [22]. (a) Conversational latency, (b) end-to-end task accuracy, and (c) MOS.

generally attempts to utilize the available bandwidth [5]. Instead, it appears to reflect an application-level design choice: the applications deliberately adopt a bitrate lower than the estimated bandwidth. However, this approach leaves a substantial portion of the available bandwidth underutilized.

Summary and Insights. Our measurement study under constrained bandwidth indicates that current applications are still at an early stage in terms of bandwidth adaptation, exhibiting significant difficulty in maintaining performance when uplink capacity is limited. Their ABR strategies often prioritize video frame rate over video quality, leading to long conversational latency and/or degraded VLM inference accuracy. Moreover, they adopt conservative sending rates that fail to fully utilize the available bandwidth. Taken together with our earlier analysis (§4.2), we argue that an effective design for video-based human–GenAI interaction should utilize available bandwidth more efficiently to sustain high video quality for VLM consumption, thereby preserving inference accuracy and, in turn, providing a good overall user experience.

5 Discussion

Inspecting Encrypted Media Pipelines. Given that all studied applications encrypt their traffic using TLS over TCP/H2 or QUIC/H3,

or DTLS over UDP/RTP, we cannot directly inspect the media payload exchanged between the user and the GenAI backend. This prevents us from knowing the exact transmitted media content, such as the video codec, encoded resolution, compression level, packetization behavior, number of frames per chunk, as well as the applications’ internal adaptation logic under degraded network conditions. In this paper, we therefore rely on traffic-level information, such as throughput, frame/chunk inter-arrival times, packet sizes, and protocol behavior, to infer transmission logic. Beyond these measurements, side-channel information from other layers of the network stack, such as IP headers [53], may also help characterize encrypted media traffic. This direction is orthogonal to our traffic-level analysis and could be combined with it to better understand the internal operation of video-based conversational GenAI applications.

Evolving Landscape. Video-based human–GenAI interaction is still at a nascent stage, with platforms rapidly iterating on both capabilities and delivery architectures. Early deployments rely on multimodal foundation models that integrate visual understanding and real-time video reasoning, which are advancing at an exceptional pace. As providers explore streaming-efficient vision–language

models and on-device inference coordination, both network behavior and user experience will evolve accordingly. Thus, our measurements capture only an early snapshot of a quickly transforming ecosystem. As these services evolve, provider-specific observations, such as protocol choices, infrastructure deployment, conversational latency, and inference accuracy, may change over time. Broader insights into the interactions among network transport, video delivery, and VLM inference may be more durable, but should still be reevaluated as models and system architectures advance. Continued longitudinal studies will be essential to validate whether current design choices persist and revisit our findings as video-based conversational GenAI systems mature.

Evaluating VLM Capability Across Model and Task Dimensions. Our measurement study focuses on deployed video-based conversational GenAI applications and evaluates their behavior mainly from a networking perspective. A complementary direction is to study how the underlying VLM’s capability affects this type of application. This can be achieved by leveraging open-source VLMs, such as Qwen3-VL [3], and evaluating variants with different model sizes (e.g., 2B, 4B, 8B, and 32B) under the same video inputs and network conditions. One challenge is that many existing VLMs are designed for finite video clips rather than continuous streaming input. Therefore, fine-tuning or adapting them to support video streaming may be necessary [68]. Another important direction is to evaluate VLMs on a broader set of task types and video content, which may exhibit different inference performance and sensitivity to video quality and frame rate. Together, these directions would complement our measurement study and provide a broader understanding of these applications from the model perspective.

Conversational Latency Decomposition. We measure conversational latency as an end-to-end user-perceived metric, from the end of the user’s audio prompt to the beginning of the GenAI audio response. This metric captures the overall responsiveness experienced by users, but it combines multiple components, such as uplink and downlink media transmission, audio processing (e.g., ASR), VLM inference, response generation, and speech synthesis. Since the applications studied are black-box systems, we cannot directly decompose latency into its individual stages. A promising direction is to design controlled experiments that isolate the contribution of each component. For instance, one could vary prompt length or phonetic difficulty while keeping the video input fixed to estimate the contribution of audio processing to the overall latency. If the measured conversational latency increases with prompt length or speech complexity, the audio-processing pipeline may be a significant contributor; otherwise, VLM inference or backend scheduling may dominate. Such latency decomposition would help identify which system components should be optimized to improve video-based human–GenAI interaction.

6 Related Work

Measurement of Real-Time Communication Applications. A rich body of work has conducted network measurement studies on various RTC systems, including video conferencing [6, 46, 57], cloud gaming [28, 60, 69], immersive telepresence systems [15], and social virtual reality platforms in the metaverse [13, 14, 39], examining different aspects such as bandwidth demand, scalability,

and resiliency. More recently, Cheng *et al.* [10] studied audio-only human–GenAI calling. In contrast, we examine a new class of RTC applications that support real-time, video-driven interaction between humans and GenAI.

Measurement Study Related to Foundation Models. Emerging measurement work has begun to examine how foundation models and their ecosystems behave in the wild. Empirical analyses of LLM application ecosystems expose pervasive data-collection practices and privacy risks [66], while measurement of human–GenAI calling services characterizes latency, traffic patterns, and service behaviors for audio-driven interactions [10]. Other studies demonstrate novel uses of language models as analysis tools, for example, leveraging transformer-based models to fingerprint Internet-connected devices from scan data [52], and systematically evaluate both personal-information extraction attacks and mitigation strategies against LLM pipelines [38].

Optimizing VLM Inference. Recent efforts to accelerate VLM inference emphasize reducing visual redundancy, improving multi-modal fusion, and enabling more efficient processing. Techniques such as SparseVILA [31] exploit visual sparsity by pruning or decoupling redundant tokens, lowering computation while preserving accuracy. Systematic investigations [33, 35] highlight key structural trade-offs, showing that inference-optimal VLMs may rely on fewer visual tokens but larger overall parameter budgets. Further optimization strategies include quantization for post-training compression [59], compatible token pruning for low-memory inference [70], and accelerated visual encoders [58]. While these advances focus on model-level optimizations of the inference pipeline, our work measures real-world video-based conversational GenAI services, providing empirical insights that complement these improvements.

7 Conclusion

In this paper, we presented the first systematic measurement study of video-based human–GenAI conversational interaction. By developing a custom toolset to analyze five leading applications with 11 variants, we provided an end-to-end characterization of their operational pipelines. Our measurements revealed substantial performance challenges, including conversational latencies well above natural turn-taking thresholds, modest and highly variable VLM inference accuracy, and significant sensitivity to choices of networking protocols and video delivery strategies. We further demonstrated that traditional video streaming heuristics, such as prioritizing stable frame rates, are often misaligned with the requirements of VLM-driven understanding, underscoring the need for a fundamental reevaluation of adaptation logic. We hope this study provides both a foundation and a roadmap for future work on building VLM-aware video delivery and network infrastructure that can support the next generation of real-time human–AI interaction.

Acknowledgment

We thank the anonymous reviewers and our shepherd for their insightful feedback, which has greatly strengthened this work. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Meta. This work was partially supported by the National Science Foundation under Grants CNS-2212296, CNS-2235049, CNS-2402992, and CNS-2437203.

References

- [1] 2026. Maximum transmission unit. https://en.wikipedia.org/wiki/Maximum_transmission_unit. [accessed on 09/01/2026].
- [2] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. <https://arxiv.org/abs/2308.12966>. [accessed on 09/01/2026].
- [3] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. 2025. Qwen3-VL Technical Report. <https://arxiv.org/pdf/2511.21631>.
- [4] ByteDance. 2025. Doubao. <https://www.doubao.com/>. [accessed on 09/01/2026].
- [5] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. 2017. Congestion Control for Web Real-time Communication. *IEEE/ACM Transactions on Networking* 25, 5 (2017), 2629–2642.
- [6] Hyunseok Chang, Matteo Varvello, Fang Hao, and Sarit Mukherjee. 2021. Can You See Me Now? A Measurement Study of Zoom, Webex, and Meet. In *Proceedings of ACM IMC*. <https://dl.acm.org/doi/abs/10.1145/3487552.3487847>
- [7] Joya Chen, Zhaoyang Lv, Shiwei Wu, Kevin Qinghong Lin, Chenan Song, Difei Gao, Jia-Wei Liu, Ziteng Gao, Dongxing Mao, and Mike Zheng Shou. 2024. VideoLLM-online: Online Video Large Language Model for Streaming Video. In *Proceedings of IEEE/CVF CVPR*.
- [8] Ruizhi Cheng, Jie Li, Songqing Chen, and Bo Han. 2025. Dissecting User Experience of Social Virtual Reality: A Tale of Five Platforms. *Proceedings of ACM on Human-Computer Interaction* 9, 7 (2025), 1–31.
- [9] Ruizhi Cheng, Erdem Murat, Lap-Fai Yu, Songqing Chen, and Bo Han. 2024. Understanding Online Education in Metaverse: Systems and User Experience Perspectives. In *Proceedings of IEEE Conference Virtual Reality and 3D User Interfaces (VR)*.
- [10] Ruizhi Cheng, Surendra Pathak, Guowu Xie, Matteo Varvello, Songqing Chen, and Bo Han. 2025. Hello, GenAI? Dissecting Human to Generative AI Calling. In *Proceedings of ACM IMC*.
- [11] Ruizhi Cheng, Mohamed Mohsin Shaik, Guowu Xie, and Bo Han. 2025. Huni: A Networked Testbed for Real-time Human and Generative AI Interaction. In *Proceedings of ACM Immercom*.
- [12] Ruizhi Cheng, Harshvardhan Takawale, Nan Wu, Nirupam Roy, Sennur Ulukus, Matteo Varvello, Eugene Chai, and Bo Han. 2026. LITE: Loss-resilient Immersive Telepresence with Multi-modal Semantics. In *Proceedings of ACM SIGCOMM*.
- [13] Ruizhi Cheng, Nan Wu, Songqing Chen, and Bo Han. 2022. Reality Check of Metaverse: A First Look at Commercial Social Virtual Reality Platforms. In *Proceedings of IEEE Metabuild*.
- [14] Ruizhi Cheng, Nan Wu, Songqing Chen, and Bo Han. 2022. Will Metaverse be NextG Internet? Vision, Hype, and Reality. *IEEE Network* 36 (2022), 197–204.
- [15] Ruizhi Cheng, Nan Wu, Matteo Varvello, Eugene Chai, Songqing Chen, and Bo Han. 2024. A First Look at Immersive Telepresence on Apple Vision Pro. In *Proceedings of ACM IMC*.
- [16] Ruizhi Cheng, Nan Wu, Matteo Varvello, Songqing Chen, and Bo Han. 2022. Are We Ready for Metaverse? A Measurement Study of Social Virtual Reality Platforms. In *Proceedings of ACM IMC*.
- [17] Ruizhi Cheng, Guowu Xie, and Bo Han. 2026. Real-time Human and Generative AI Interaction: Network Challenges and Opportunities. *IEEE Network* (2026).
- [18] Wesley Eddy. 2022. Transmission Control Protocol (TCP). RFC 9293. [accessed on 09/01/2026]. <https://datatracker.ietf.org/doc/html/rfc9293>
- [19] ElevenLabs. 2025. ElevenLabs: Text to Speech & AI Voice Generator. <https://elevenlabs.io/>. [accessed on 09/01/2026].
- [20] Genymotion. 2025. Genymotion - Android Emulator in the Cloud and for PC Mac. <https://www.genymotion.com/>. [accessed on 09/01/2026].
- [21] Moinak Ghoshal, Imran Khan, Z Jonny Kong, Phuc Dinh, Jiayi Meng, Y Charlie Hu, and Dimitrios Koutsonikolas. 2023. Performance of Cellular Networks on the Wheels. In *Proceedings of ACM IMC*.
- [22] Utkarsh Goel, Moritz Steiner, Mike P Wittie, Stephen Ludin, and Martin Flack. 2017. Domain-Sharding for Faster HTTP/2 in Lossy Cellular Networks. <https://arxiv.org/pdf/1707.05836>. [accessed on 09/01/2026].
- [23] Google. 2025. Gemini. <https://gemini.google.com/>. [accessed on 09/01/2026].
- [24] Google. 2026. Gemini API for Video Understanding. <https://ai.google.dev/gemini-api/docs/video-understanding>. [accessed on 09/01/2026].
- [25] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. 2022. Ego4D: Around the World in 3,000 Hours of Egocentric Video. In *Proceedings of IEEE/CVF CVPR*.
- [26] Sangtae Ha, Injong Rhee, and Lisong Xu. 2008. CUBIC: A New TCP-Friendly High-Speed TCP Variant. *ACM SIGOPS operating systems review* 42, 5 (2008), 64–74.
- [27] ipinfo.io. 2026. <https://ipinfo.io/>. [accessed on 09/01/2026].
- [28] Hassan Iqbal, Ayesha Khalid, and Muhammad Shahzad. 2021. Dissecting Cloud Gaming Performance with DECAF. In *Proceedings of ACM Measurement and Analysis of Computing Systems (SIGMETRICS)*.
- [29] Jana Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000. [accessed on 09/01/2026]. <https://datatracker.ietf.org/doc/html/rfc9000>
- [30] Junchen Jiang, Shijie Sun, Vyas Sekar, and Hui Zhang. 2017. Pytheas: Enabling Data-Driven Quality of Experience Optimization Using Group-Based Exploration-Exploitation. In *Proceedings of NSDI*.
- [31] Samir Khaki, Junxian Guo, Jiaming Tang, Shang Yang, Yukang Chen, Konstantinos N. Platanotis, Yao Lu, Song Han, and Zhijian Liu. 2025. SparseVILA: Decoupling Visual Sparsity for Efficient VLM Inference. In *Proceedings of IEEE/CVF ICCV*.
- [32] Sehoon Kim, Amir Gholami, Albert Shaw, Nicholas Lee, Karttikeya Mangalam, Jitendra Malik, Michael W. Mahoney, and Kurt Keutzer. 2022. Squeezeformer: An Efficient Transformer for Automatic Speech Recognition. In *Proceedings of NeurIPS*.
- [33] Hugo Laurençon, Léo Tronchon, Matthieu Cord, and Victor Sanh. 2024. What Matters When Building Vision-Language Models?. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*.
- [34] Kunchang Li, Yali Wang, Yanan He, Yizhuo Li, Yi Wang, Yi Liu, Zun Wang, Jilan Xu, Guo Chen, Ping Luo, et al. 2024. MVBench: A Comprehensive Multi-modal Video Understanding Benchmark. In *Proceedings of IEEE/CVF CVPR*.
- [35] Kevin Y. Li, Sachin Goyal, Joao D. Semedo, and J. Zico Kolter. 2025. Inference-Optimal VLMs Need Fewer Visual Tokens and More Parameters. In *Proceedings of ICLR*.
- [36] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. 2020. Reducto: On-Camera Filtering for Resource-Efficient Real-Time Video Analytics. In *Proceedings of ACM SIGCOMM*.
- [37] Zhihao Li, Dave Levin, Neil Spring, and Bobby Bhattacharjee. 2018. Internet AnyCast: Performance, Problems, & Potential. In *Proceedings of ACM SIGCOMM*.
- [38] Yupei Liu, Yuqi Jia, Jinyuan Jia, and Neil Zhenqiang Gong. 2025. Evaluating LLM-based Personal Information Extraction and Countermeasures. In *Proceedings of USENIX Security Symposium*.
- [39] Minzhao Lyu, Rahul Dev Tripathi, and Vijay Sivaraman. 2023. Metavradar: Measuring Metaverse Virtual Reality Network Activity. In *Proceedings of ACM SIGMETRICS*. <https://doi.org/10.1145/3626786>
- [40] Kyle MacMillan, Tarun Mangla, James Saxon, and Nick Feamster. 2021. Measuring the Performance and Network Utilization of Popular Video Conferencing Applications. In *Proceedings of ACM IMC*.
- [41] Isaia Majil, Mau-Tsuen Yang, and Sophia Yang. 2022. Augmented Reality Based Interactive Cooking Guide. *Sensors* 22, 21 (2022), 8290.
- [42] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural Adaptive Video Streaming with Pensieve. In *Proceedings of ACM SIGCOMM*. <https://dl.acm.org/doi/10.1145/3098822.3098843>
- [43] MaxMind. 2026. <https://www.maxmind.com/en/home>. [accessed on 09/01/2026].
- [44] Trevor Mendez, Walter Milliken, and Dr. Craig Partridge. 1993. Host Anycasting Service. RFC 1546. [accessed on 09/01/2026]. <https://www.rfc-editor.org/info/rfc1546>
- [45] Zili Meng, Yaning Guo, Chen Sun, Bo Wang, Justine Sherry, Hongqiang Harry Liu, and Mingwei Xu. 2022. Achieving Consistent Low Latency for Wireless Real-Time Communications with the Shortest Control Loop. In *Proceedings of ACM SIGCOMM*.
- [46] Oliver Michel, Satadal Sengupta, Hyojoon Kim, Ravi Netravali, and Jennifer Rexford. 2022. Enabling Passive Measurement of Zoom Performance in Production Networks. In *Proceedings of ACM IMC*.
- [47] OpenAI. 2024. Be My Eyes Accessibility with GPT-4o. <https://www.youtube.com/watch?v=KwNUJ69RbWY&t=1s>. [accessed on 09/01/2026].
- [48] OpenAI. 2025. ChatGPT. <https://openai.com/index/chatgpt/>. [accessed on 09/01/2026].
- [49] OpenAI. 2025. The next chapter of the Microsoft–OpenAI partnership. <https://tinyurl.com/bd8zrne>. [accessed on 09/01/2026].
- [50] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of ICML*.
- [51] Yi Ren, Yangjun Ruan, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. 2019. FastSpeech: Fast, Robust and Controllable Text to Speech. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*.
- [52] Armin Sarabi, Tongxin Yin, and Mingyan Liu. 2023. An LLM-based Framework for Fingerprinting Internet-connected Devices. In *Proceedings of ACM Internet Measurement Conference (IMC)*.
- [53] Taveesh Sharma, Tarun Mangla, Arpit Gupta, Junchen Jiang, and Nick Feamster. 2023. Estimating WebRTC Video QoE Metrics Without Using Application Headers. In *Proceedings of ACM IMC*.
- [54] SoloX. 2025. SoloX - Real-time collection tool for Android/iOS performance data. <https://github.com/smart-test-ti/SoloX>. [accessed on 09/01/2026].
- [55] Bruce Spang, Shravya Kunamalla, Renata Teixeira, Te-Yuan Huang, Grenville Armitage, Ramesh Johari, and Nick McKeown. 2023. Sammy: Smoothing Video Traffic to Be a Friendly Internet Neighbor. In *Proceedings of ACM SIGCOMM*.
- [56] Tencent. 2025. Yuanbao. <https://yuanbao.tencent.com/>. [accessed on 09/01/2026].

- [57] Matteo Varvello, Hyunseok Chang, and Yasir Zaki. 2022. Performance Characterization of Videoconferencing in the Wild. In *Proceedings of ACM IMC*. <https://doi.org/10.1145/3517745.3561442>
- [58] Pavan Kumar Anasosalu Vasu, Fartash Faghri, Chun-Liang Li, Cem Koc, Nate True, Albert Antony, Gokul Santhanam, James Gabriel, Peter Grasch, Oncel Tuzel, and Hadi Pouransari. 2025. FastVLM: Efficient Vision Encoding for Vision-Language Models. In *Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [59] Changyuan Wang, Ziwei Wang, Xiuwei Xu, Yansong Tang, Jie Zhou, and Jiwen Lu. 2024. Q-VLM: Post-training Quantization for Large Vision-Language Models. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*.
- [60] Yifan Wang, Minzhao Lyu, and Vijay Sivaraman. 2025. Games Are Not Equal: Classifying Cloud Gaming Contexts for Effective User Experience Measurement. In *Proceedings of ACM IMC*.
- [61] Whois. 2026. <https://www.whois.com/whois/>. [accessed on 09/01/2026].
- [62] Wikipedia. 2025. Big Tech. https://en.wikipedia.org/wiki/Big_Tech. [accessed on 09/01/2026].
- [63] Wireshark. 2026. https://www.wireshark.org/docs/wsug_html_chunked/ChAdvTCPAnalysis.html. [accessed on 09/01/2026].
- [64] Jiangkai Wu, Zhiyuan Ren, Liming Liu, and Xinggong Zhang. 2025. Chat with AI: The Surprising Turn of Real-time Video Communication from Human to AI. In *Proceedings of ACM HotNets*.
- [65] Kui Wu, Shuhang Xu, Hao Chen, Churan Wang, Zhoujun Li, Yizhou Wang, and Fangwei Zhong. 2025. VLM Can Be a Good Assistant: Enhancing Embodied Visual Tracking with Self-Improving Visual-Language Models. In *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [66] Yuhao Wu, Evin Jaff, Ke Yang, Ning Zhang, and Umar Iqbal. 2025. An In-Depth Investigation of Data Collection in LLM App Ecosystems. In *Proceedings of ACM Internet Measurement Conference (IMC)*.
- [67] xAI. 2025. Grok. <https://x.ai/>. [accessed on 09/01/2026].
- [68] Ruyi Xu, Guangxuan Xiao, Yukang Chen, Liuning He, Kelly Peng, Yao Lu, and Song Han. 2025. StreamingVLM: Real-time understanding for infinite video streams. <https://arxiv.org/abs/2510.09608>.
- [69] Xiaokun Xu and Mark Claypool. 2022. Measurement of Cloud-based Game Streaming System Response to Competing TCP Cubic or TCP BBR Flows. In *Proceedings of ACM IMC*.
- [70] Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Chendi Li, Jinghua Yan, Yu Bai, Ponnuswamy Sadayappan, Xia Hu, and Bo Yuan. 2025. TopV: Compatible Token Pruning with Inference Time Optimization for Fast and Low-Memory Multimodal Vision-Language Model. In *Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [71] Dong Yu and Lin Deng. 2016. *Automatic Speech Recognition*. Vol. 1. Springer.
- [72] Yuhang Zang, Wei Li, Jun Han, Kaiyang Zhou, and Chen Change Loy. 2025. Contextual Object Detection with Multimodal Large Language Models. *International Journal of Computer Vision* 133, 2 (2025), 825–843.
- [73] Jingyi Zhang, Jiaxing Huang, Sheng Jin, and Shijian Lu. 2024. Vision-Language Models for Vision Tasks: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 8 (2024), 5625–5644.